Zofia Kapcińska 160227

Sprawozdanie: Tryby pracy szyfrów blokowych

1. Screenshot.

```python
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.backends import default_backend
import time
import os

def cipherMode(mode, iv, nonce):  2 usages  kwonbx
    match mode:
        case 'ECB': return modes.ECB()
        case 'CBC': return modes.CBC(iv)
        case 'CFB': return modes.CFB(iv)
        case 'OFB': return modes.OFB(iv)
        case 'CTR': return modes.CTR(nonce)
        case _: return None

def benchmark(modeNames, key, iv, nonce):  1 usage  kwonbx
    files = {
        "10MB": 1024*1024*10,
        "100MB": 1024*1024*100,
        "500MB": 1024*1024*500,
    }

    for label, bytes in files.items():
        data = os.urandom(bytes)
        padder = padding.PKCS7(128).padder()
        paddedData = padder.update(data) + padder.finalize()

        print("\nWielkość danych: " + label)

        for mode in modeNames:
            modeObject = cipherMode(mode, iv, nonce)
            cipher = Cipher(algorithms.AES(key), modeObject, backend=default_backend())

            encryptor = cipher.encryptor()
            startEnc = time.time()
            cipherText = encryptor.update(paddedData) + encryptor.finalize()
            endEnc = time.time()

            decryptor = cipher.decryptor()
            startDec = time.time()
            decryptedText = decryptor.update(cipherText) + decryptor.finalize()
            endDec = time.time()

            totalEnc = endEnc - startEnc
            totalDec = endDec - startDec

            print(mode + ":")
            print("Czas szyfrowania:" + str(totalEnc) + " | Czas deszyfrowania: " + str(totalDec))
```

```python
def errorPropagation(modeNames, key, iv, nonce):  1 usage  kwonbx
    plainText = b"Wiadomosc przygotowana na potrzeby testu. ABCDEFGHIJKLMNOPQRSTUVWXYZ123456789"
    plainText = plainText[:64]

    for mode in modeNames:
        modeObject = cipherMode(mode, iv, nonce)
        cipher = Cipher(algorithms.AES(key), modeObject, backend=default_backend())

        encryptor = cipher.encryptor()
        cipherText = encryptor.update(plainText) + encryptor.finalize()
        corruptedText = bytearray(cipherText)
        corruptedText[20] = corruptedText[20] ^ 0xFF

        decryptor = cipher.decryptor()
        decryptedText = decryptor.update(corruptedText) + decryptor.finalize()
        diffs = [i for i in range(len(plainText)) if plainText[i] != decryptedText[i]]

        if len(diffs) == 1:
            print(mode + ": Tylko 1 bajt został zmieniony")
        elif len(diffs) <= 16:
            print(mode + ": Błąd w obrębie 1 bloku (zmieniono " + str(len(diffs)) + " bajtów)")
        else:
            print(mode + ": Propagacja! Uszkodzono " + str(len(diffs)) + " bajtów")

def xor(b1, b2):  2 usages  kwonbx
    return bytes(a ^ b for a, b in zip(b1, b2))

def cbcEncrypt(text, key, iv):  1 usage  kwonbx
    cipher = Cipher(algorithms.AES(key), modes.ECB(), backend=default_backend())
    encryptor = cipher.encryptor()
    padder = padding.PKCS7(128).padder()
    paddedData = padder.update(text) + padder.finalize()

    cipherText = b""
    prevBlock = iv

    for i in range(0, len(paddedData), 16):
        currentBlock = paddedData[i:i + 16]
        blockToEncrypt = xor(currentBlock, prevBlock)
        encryptedBlock = encryptor.update(blockToEncrypt)
        cipherText += encryptedBlock
        prevBlock = encryptedBlock

    return cipherText
```

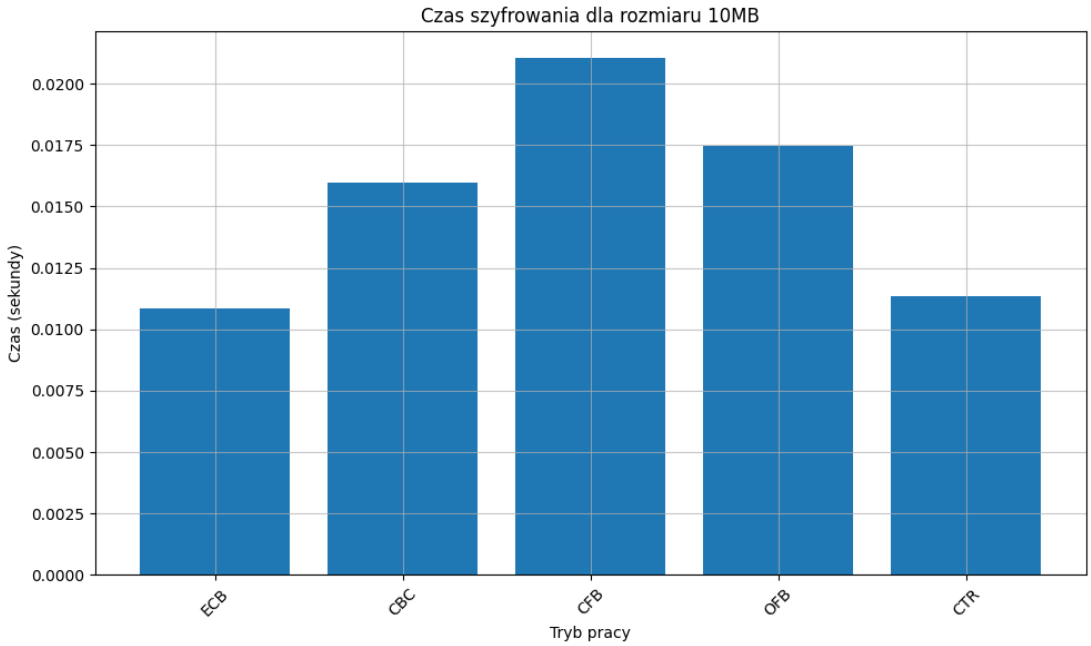
```python
def cbcDecrypt(cipherText, key, iv):  1 usage  kwonbx
    cipher = Cipher(algorithms.AES(key), modes.ECB(), backend=default_backend())
    decryptor = cipher.decryptor()

    decryptedText = b""
    prevBlock = iv

    for i in range(0, len(cipherText), 16):
        currentBlock = cipherText[i:i + 16]
        decryptedBlock = decryptor.update(currentBlock)
        plainBlock = xor(decryptedBlock, prevBlock)
        decryptedText += plainBlock
        prevBlock = currentBlock

    unpadder = padding.PKCS7(128).unpadder()
    return unpadder.update(decryptedText) + unpadder.finalize()


if __name__ == "__main__":
    modeNames = ["ECB", "CBC", "CFB", "OFB", "CTR"]
    key = os.urandom(16)
    iv = os.urandom(16)
    nonce = os.urandom(16)
    benchmark(modeNames, key, iv, nonce)
    print("")
    errorPropagation(modeNames, key, iv, nonce)

    msg = "Wiadomosc przygotowana na potrzeby testu. (CBC)".encode("utf-8")
    enc = cbcEncrypt(msg, key, iv)
    dec = cbcDecrypt(enc, key, iv)
    print("\nWiadomość do zaszyfrowania: " + str(msg))
    print("Wiadomość zaszyfrowana: " + str(enc))
    print("Wiadomość zdeszyfrowana: " + str(dec))
```

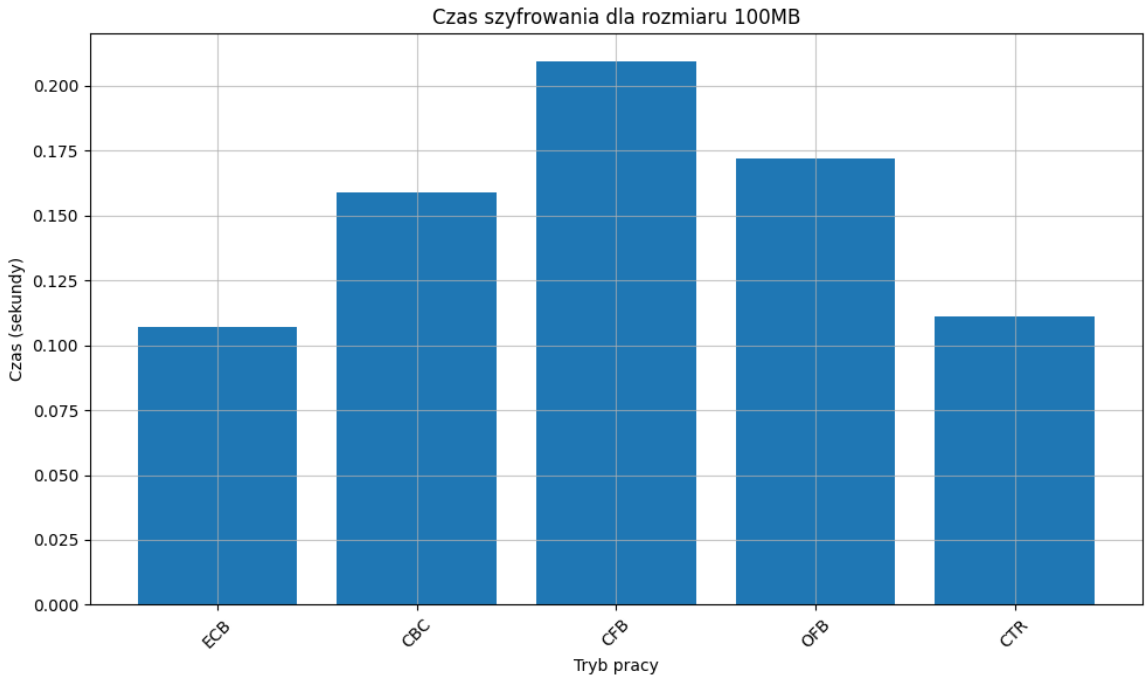
2. Zestawienie wyników wraz ze stosownymi wnioskami.

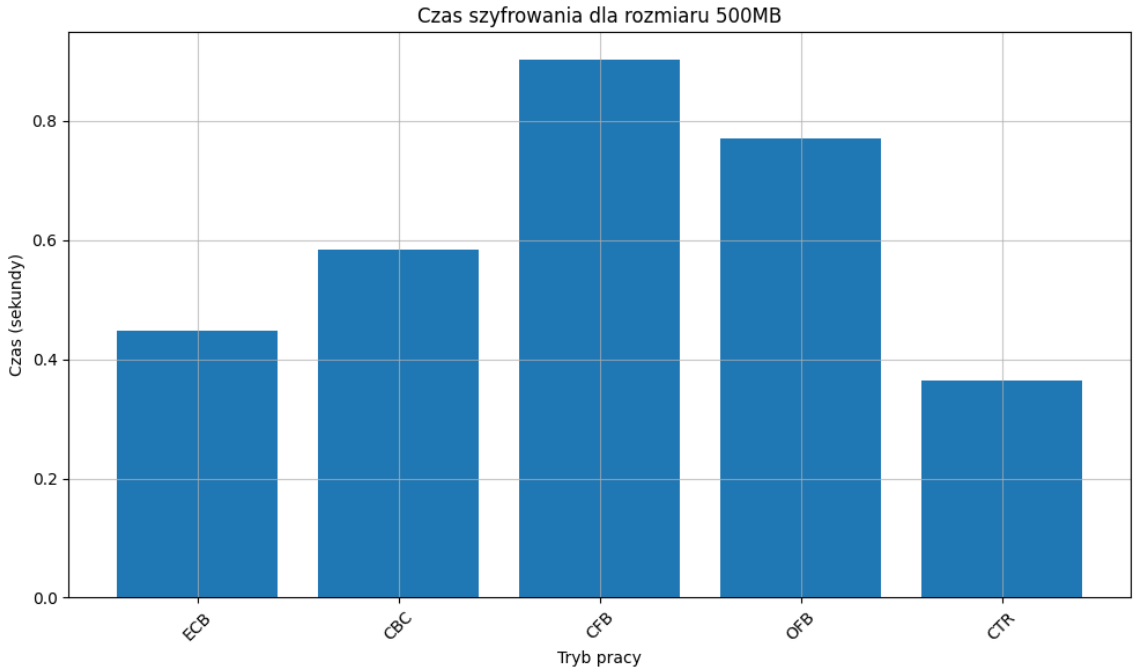
1) Porównanie algorytmów

Szyfrowanie:

Tryb pracy	Czas działania (10 MB) [s]	Czas działania (100 MB) [s]	Czas działania (500 MB) [s]
ECB	0.01085	0.10720	0.44838
CBC	0.01596	0.15883	0.58389
CFB	0.02106	0.20964	0.90355
OFB	0.01748	0.17193	0.77076
CTR	0.01133	0.11130	0.36478

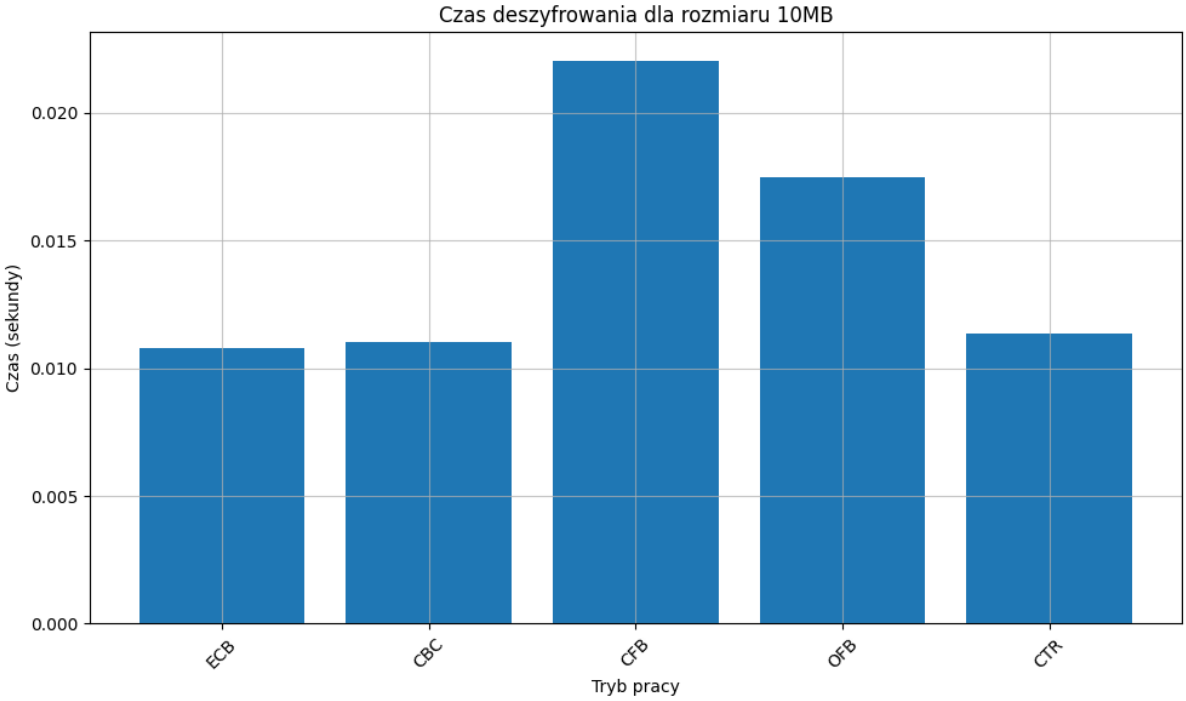


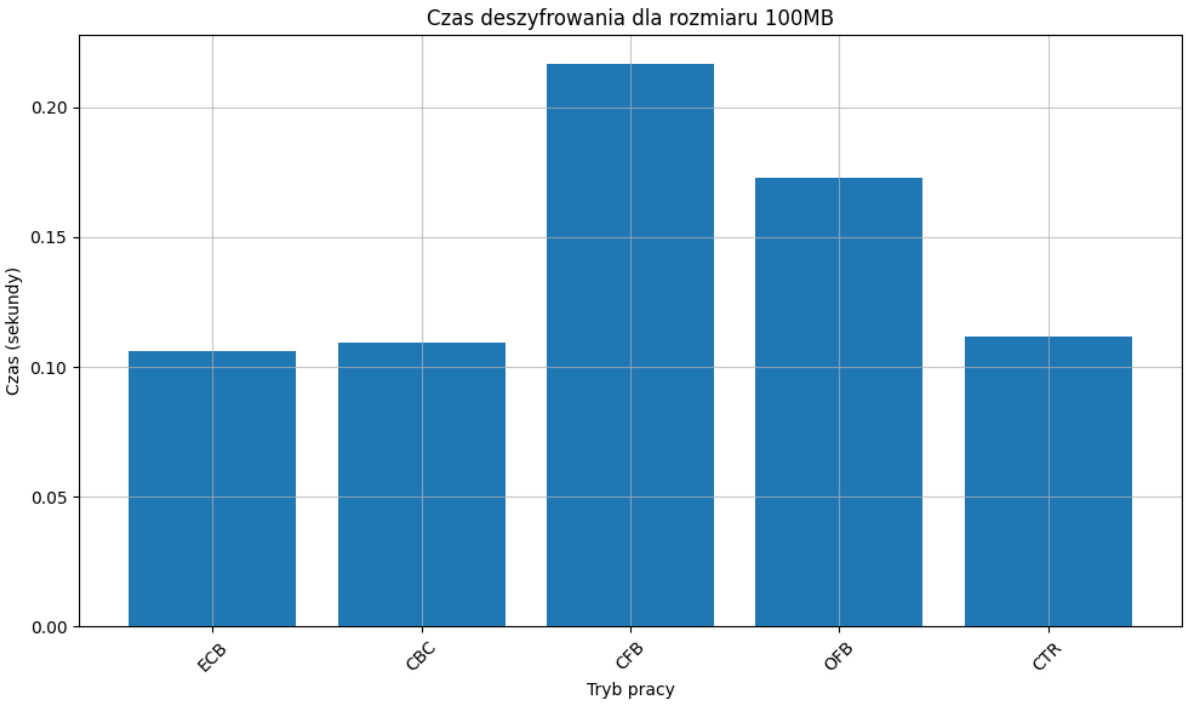

Czas szyfrowania dla rozmiaru 100MB
Czas (sekundy)
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
ECB
CBC
CFB
OFB
CTR
Tryb pracy

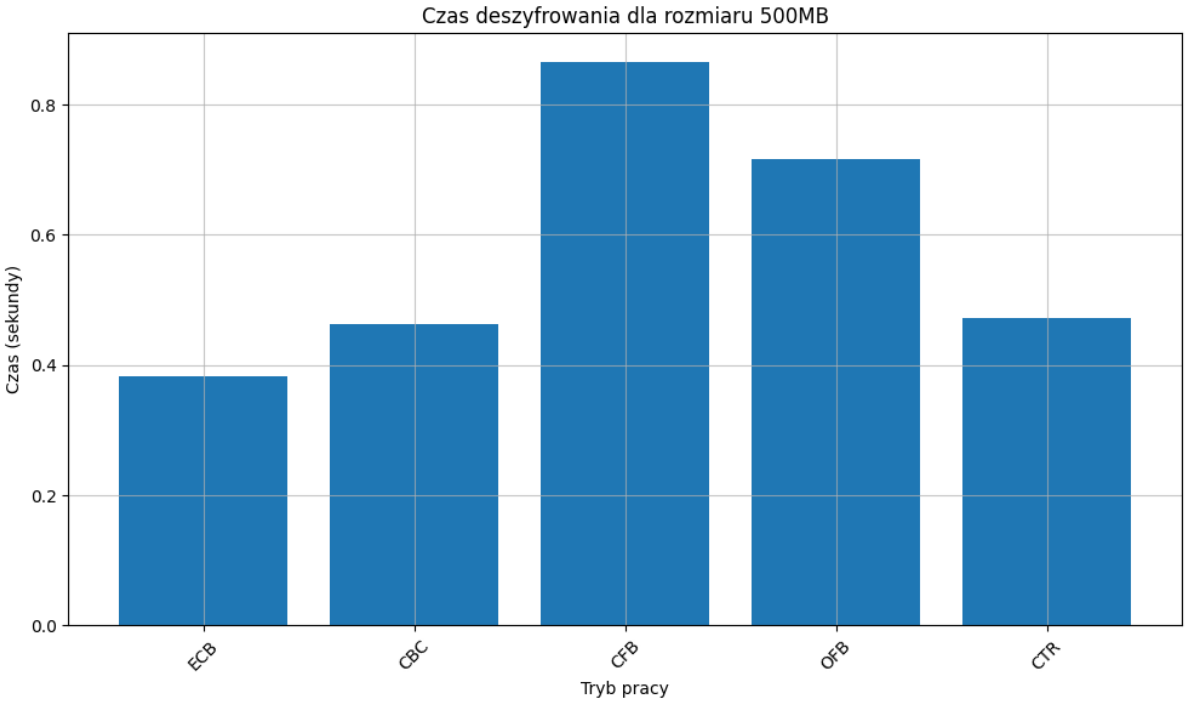

Czas szyfrowania dla rozmiaru 500MB
Czas (sekundy)
0.0
0.2
0.4
0.6
0.8
ECB
CBC
CFB
OFB
CTR
Tryb pracy

Deszyfrowanie:

Tryb pracy	Czas działania (10 MB) [s]	Czas działania (100 MB) [s]	Czas działania (500 MB) [s]
ECB	0.01076	0.10634	0.38361
CBC	0.01101	0.10928	0.46343
CFB	0.02205	0.21693	0.86609
OFB	0.01749	0.17287	0.71619
CTR	0.01136	0.11183	0.47199




Czas deszyfrowania dla rozmiaru 100MB
Czas (sekundy)
0.00
0.05
0.10
0.15
0.20
ECB
CBC
CFB
OFB
CTR
Tryb pracy


Czas deszyfrowania dla rozmiaru 500MB
Czas (sekundy)
0.0
0.2
0.4
0.6
0.8
ECB
CBC
CFB
OFB
CTR
Tryb pracy

2) Propagacja błędów
   ECB: Błąd w obrębie 1 bloku (zmieniono 15 bajtów)
   CBC: Propagacja! Uszkodzono 17 bajtów
   CFB: Propagacja! Uszkodzono 17 bajtów
   OFB: Tylko 1 bajt został zmieniony
   CTR: Tylko 1 bajt został zmieniony

3) Implementacja CBC

   Wiadomość do zaszyfrowania: b'Wiadomosc przygotowana na potrzeby testu. (CBC)'

   Wiadomość zaszyfrowana: b'Y&;\xa1\x9bY\x81\x01\xcfR\xb5\\\x9e\xbc\xf5\xe7s\xcd8QC-\xe3\xae\xb5c/\xceDi\xdf\xec\x03\x01I\x04b-\x18U\xe3\x0b\xa7\n\xe3\xa3P\x06'

   Wiadomość zdeszyfrowana: b'Wiadomosc przygotowana na potrzeby testu. (CBC)'

Wnioski:

Analiza uzyskanych wyników pozwala stwierdzić, że wydajność poszczególnych trybów pracy szyfrów blokowych jest ściśle powiązana z ich architekturą oraz stopniem skomplikowania. Najwyższą efektywnością wykazują się tryby ECB oraz CTR, które osiągają najkrótsze czasy zarówno podczas szyfrowania, jak i deszyfrowania, co wynika z braku zależności między kolejnymi blokami i możliwości równoległego przetwarzania operacji. W przeciwieństwie do nich tryby CFB oraz OFB są zdecydowanie wolniejsze. Ciekawym przypadkiem jest tryb CBC, gdzie deszyfrowanie przebiega szybciej niż szyfrowanie. Wszystkie testowane tryby wykazują liniową zależność czasu pracy od wielkości danych.